

ai-os-new

Top 50 Deduplicated Bug Priorities

A cross-audit analysis of 692 findings from 10 independent audits, deduplicated by root cause and prioritized by severity, cross-audit frequency, and blast radius.

Metric	Value
Total raw findings across 10 audits	692
Estimated unique root causes (after dedup)	~300
Top 50 priorities in this report	50
Critical root causes (fix first)	15
Audits contributing findings	Leak, Race, State, UX, Observability, Data Integrity, Contract, Logic Bugs, Type Contract, Build/Deploy

Methodology

Ten independent audits were conducted on the ai-os-new codebase, each focusing on a different aspect: memory/resource leaks, race conditions/lifecycle, state inconsistency, UX/correctness, observability/monitoring, data integrity/persistence, contract violations, logic bugs, type/contract mismatches, and build/deploy/config. Together they produced 692 raw findings. Many findings describe the same root cause from different angles. This report deduplicates them by identifying the underlying root cause, then prioritizes by three criteria:

1. Cross-audit frequency — How many independent audits flagged the same root cause? A bug seen in 5 audits is more systemic than one seen in 1. 2. Severity — The highest severity assigned across all audits. CRITICAL means data loss, security bypass, or production outage. 3. Blast radius — How many files, services, or components are affected? A bug in a utility used by 30 services is more impactful than one in a single panel.

Fixing the top 15 critical root causes alone would eliminate approximately 40% of all 692 findings, because they are the upstream sources that cascade into dozens of downstream symptoms. The recommended approach is: fix Tier 1 (ranks 1-10) first, then Tier 2 (11-20), then Tier 3 (21-35), then Tier 4 (36-50).

Severity Legend

Level	Meaning	Fix Timeline
CRITICAL	Data loss, security bypass, or production outage	Immediate (next sprint)
HIGH	Incorrect behavior, broken feature, or misleading state	Within 2 sprints
MEDIUM	Degraded experience, stale data, or missing validation	Within 1 month

#1	CRITICAL	5 audits	Silent catch blocks swallow all errors across 20+ services
Root cause: Over 20 catch blocks across the codebase silently swallow errors with empty catch, catch that returns null, or catch that only logs to console. This makes production debugging nearly impossible and hides data loss, state corruption, and security failures.			
Affected files: health-service.ts, storage-adapter.ts, cross-tab-state.ts, config-service.ts, event-recorder.ts, settings-service.ts, virtual-key-service.ts, external-secrets-service.ts, migration-control-layer.ts, agent-service.ts, snapshot-service.ts, and 10+ more			
Safe fix: Add structured ILogger calls to every catch block. At minimum: logger.error(context, error). For critical paths, also emit an event to the event bus so monitoring can detect patterns.			
Original findings: OBS-01/02/03/71/91/94/95/101/102/103/107/108, CV-17/18, TC-30/31/32, UX-09, BLD-10			
#2	CRITICAL	5 audits	Key status triple-replicated with semantic mismatch and no cascade cleanup
Root cause: Key status lives in KeyRegistry, KeyStateStore, and useKeyStore with different enum values and no synchronization. Deleting a key from KeyService does not cascade to KeyLifecycle, KeyHealth, KeyRotationPolicy, VirtualKeyService, or NoteRepository internal Maps, leaving dangling references and phantom keys.			
Affected files: key-registry.ts, key-state-store.ts, useKeyStore.ts, key-service.ts, key-lifecycle.ts, key-health.ts, key-rotation-policy.ts, virtual-key-service.ts, note-repository.ts			
Safe fix: 1) Make KeyService.deleteKey() call cleanup on all 6 internal Maps. 2) Unify KeyStatus/KeyState into a single enum. 3) Add event KEY_DELETED that all services subscribe to for cascade cleanup.			
Original findings: SI-01/08/09/10/11/12, OBS-41/49/55, CV-12/13/29, TC-12/24, D-10/11			

#3	CRITICAL	4 audits	Zero event emissions on critical lifecycle transitions
Root cause: Key lifecycle state transitions (active->degraded->quarantined), DebateRoom lifecycle operations, debate branching (fork/merge/rollback), vault lock/unlock/purge, and config mutations all emit zero events to the event bus. This makes the system unobservable and audit-impossible. Affected files: key-lifecycle.ts, debate-room.ts, debate-branching.ts, key-vault.ts, config-registry.ts, feature-flag-service.ts, debate-memory.ts, debate-consensus.ts, debate-evaluator.ts Safe fix: Add event emission at every lifecycle transition. Use existing event constants where available; define new ones where not. Follow the pattern: <code>this.eventBus.emit(EVENT_NAME, { correlationId, ...payload })</code>. Original findings: OBS-41/42/43/44/45/46/47/48/49/50/51/52/55/56/57/61/62, OBS-92/93			
#4	CRITICAL	4 audits	Event payload mismatches across EventMap, DomainEventMap, and actual emits
Root cause: Over 15 event types have payloads that completely disagree between their TypeScript type definitions (EventMap, DomainEventMap, CognitiveEventMap) and actual runtime emission. This causes silent data loss, type casts, and runtime crashes when subscribers receive unexpected shapes. Affected files: event-bus.ts, domain-events.ts, cognitive-events.ts, chat-events.ts, provider-events.ts, observability-events.ts, settings-service.ts, chat-service.ts, role-service.ts, tool-executor.ts Safe fix: 1) Run automated payload shape comparison at startup (dev mode). 2) For each mismatched event, align actual emit with the most complete type definition. 3) Add runtime validation with Zod schemas. Original findings: TC-01/02/03/04/05/06/07/08/22, CV-05/06/22			
#5	CRITICAL	4 audits	useEffect cleanup missing: timers, streams, listeners leak on unmount
Root cause: Over 30 React components have useEffect hooks that start timers, recursive setTimeout loops, fetch requests, or event listeners without cleanup return functions. When components unmount, zombie callbacks fire, causing setState on unmounted components, memory leaks, and stale data mutations. Affected files: DebateWorkspacePanel.tsx, SREAgentPanel.tsx, VoiceButton.tsx, CodeRunner.tsx, HealthPanel.tsx, DocsHealthPanel.tsx, DebatePanel.tsx, EventsTimeline.tsx, AgentsPanelContainer.tsx, AquariumPanel hooks Safe fix: Add cleanup return functions to all useEffect hooks. For timers: <code>return () => clearTimeout(id)</code>. For fetches: use AbortController and abort on cleanup. For recursive loops: set a cancelled flag. Original findings: L-01/04/08/10/11/14/15/16/24/25/26/27/33/34/35/36, RC-11/12/22/23/36/37/47			
#6	CRITICAL	4 audits	Cross-tab state sync can silently desync with no detection
Root cause: CrossTabStateSync overwrites fresh state with stale data (no timestamp ordering), primary election always returns true (base-36 vs base-10 parsing), broadcast injects fabricated data, and there is no checksum or heartbeat to detect when tabs diverge. Affected files: cross-tab-state.ts Safe fix: 1) Add Lamport timestamps to all sync messages. 2) Fix primary election: use <code>parseInt(id, 36)</code>. 3) Add periodic state hash comparison between tabs. 4) Emit SYNC_DESYNC event when hashes diverge. Original findings: SI-02/21/43, RC-09/10, OBS-72, CV-10/56			

#7

CRITICAL

4 audits

Four independent provider health tracking systems can disagree

Root cause: HealthService, HealthScoreService, ProviderTracker, and MonitoringService each track provider health independently with different schemas, thresholds, and data sources. Unknown providers default to 'healthy' in some but not others, creating phantom healthy signals that mislead routing.

Affected files: health-service.ts, health-score-service.ts, provider-tracker.ts, monitoring-service.ts, provider-instance.ts, provider-state.ts

Safe fix: 1) Make ProviderTracker the single source of truth for provider health. 2) HealthService reads from ProviderTracker instead of computing independently. 3) Unknown providers must return status:'unknown' not 'healthy'.

Original findings: SI-26, OBS-06/18/19/22/25/26/27, CV-27/29, LG-27

#8

CRITICAL

4 audits

Config cascade across four systems loses updates and creates split-brain

Root cause: ConfigRegistry global CONFIG, ConfigService overlays, SettingsService, and RouterConfigManager each hold separate config state. ConfigService overlays merge against already-mutated CONFIG (not defaults), ConfigHistoryService.rollback doesn't clear overlays, and SettingsService uses shallow merge that loses new nested fields.

Affected files: config-registry.ts, config-service.ts, settings-service.ts, router-config-manager.ts, config-history.ts

Safe fix: 1) Freeze CONFIG defaults at startup; never mutate the global. 2) ConfigService overlays must always merge against the frozen default. 3) ConfigHistoryService.rollback must clear overlays. 4) Use deep merge for SettingsService.

Original findings: SI-27, BLD-03/04/15/18/31, CV-28, TC-37

#9

CRITICAL

4 audits

Cache invalidation failures: stale data persists across 12+ services

Root cause: SnapshotService.restore() does not invalidate caches or projections. CacheDecorator accumulates expired entries that are never deleted. Key state changes don't invalidate dependent caches. CacheService persist swallows write failures. Cache stampede on concurrent misses.

Affected files: cache-service.ts, cache-decorator.ts, snapshot-service.ts, key-state-store.ts, provider-tracker.ts, pricing-service.ts, memory-engine.ts

Safe fix: 1) Add CacheInvalidator service that all state-mutating events trigger. 2) Proactive TTL eviction on CacheDecorator. 3) SnapshotService.restore() calls invalidateAll(). 4) Use write-through or at least log persist failures.

Original findings: SI-04/16/32, D-11/12/13/30/31/32/33/34/35, CV-15/45, TC-33

#10

CRITICAL

3 audits

Cumulative counters masquerade as health metrics: health can never recover

Root cause: ProviderInstance.getHealth(), CacheService hit/miss counters, ProviderRuntimeState.globalErrorRate, and BudgetService.agentSpend all use lifetime cumulative counters. A provider that had errors in its first hour shows as unhealthy forever, even with 100% success for days afterward.

Affected files: provider-instance.ts, cache-service.ts, provider-state.ts, budget-service.ts, key-lifecycle.ts, key-analytics.ts

Safe fix: Replace cumulative counters with sliding window metrics (e.g., last 5 minutes or exponential moving average). Add reset or decay mechanisms. Window size should be configurable.

Original findings: OBS-19/20/26/32, CV-04, LG-33

#11

CRITICAL

3 audits

Debate state in 4+ independent locations that never reconcile

Root cause: Debate state lives in DebateService, DebateEngine, DebateRoom, debateLiveStore, DebatePanel, debate-state.ts, and debate-runtime-state.ts. None is the authoritative source. Phase models are incompatible across files. Paused sessions can never resume.

Affected files: debate-service.ts, debate-engine.ts, debate-room.ts, debateLiveStore.ts, debate-session.ts, debate-bridge.ts, debate-state.ts

Safe fix: 1) Make DebateEngine the single source of truth. 2) All other locations read from it via events. 3) Unify DebatePhase enum across all files. 4) Fix VALID_TRANSITIONS to include paused->queued.

Original findings: SI-03/48, OBS-42/43, CV-02/03/21/25, LG-05/15/46

#12

CRITICAL

3 audits

Budget system: three independent trackers disagree on whether a request is within budget

Root cause: BudgetService, ProviderBudget, and quota-state.ts track spending independently. BudgetService double-counts current request cost. Cost Manager treats totalTokens as outputTokens (3x inflation). UsageTracker.checkQuota() always succeeds. Orchestration rate limiter compares token count to monetary cost.

Affected files: budget-service.ts, provider-budget.ts, cost-manager.ts, usage-tracker.ts, orchestration-service.ts, key-quotas.ts, pricing-service.ts

Safe fix: 1) Make BudgetService the single budget authority. 2) Fix double-counting in threshold check. 3) Fix totalTokens vs outputTokens in CostManager. 4) Make checkQuota() actually enforce. 5) Add unit dimension checks.

Original findings: SI-25, LG-01/02/10/12/14/20/21/34/50, OBS-17/21/40, CV-37

#13

CRITICAL

3 audits

Bootstrap ordering: EventBridge starts after bootstrap events, services miss lifecycle

Root cause: LifecycleManager.startAll() is never called. EventBridge initializes after bootstrap events are emitted, so projections miss all startup events. KEYS_LOADED fires before subscribers exist. Post-initServices services are not in LifecycleManager. Bootstrap overwrites Phase4 orchestrator with a discarded instance.

Affected files: bootstrap.ts, lifecycle-manager.ts, event-bridge.ts, key-storage-hydrator.ts, phase4-agents-roles.ts

Safe fix: 1) Call lifecycleManager.startAll() in bootstrap. 2) Initialize EventBridge before any events are emitted. 3) Register all services with LifecycleManager before startup.

Original findings: RC-01/02/07/08/30, OBS-40, BLD-27/35/40

#14

CRITICAL

3 audits

nginx/Docker: missing LLM routes, broken env vars, container won't start in production

Root cause: The nginx config uses \${API_UPSTREAM} but envsubst never runs, so the container fails to start. All /proxy/* LLM routes are missing from nginx, causing 404s for every LLM call in production. CSP in production removes 'unsafe-eval', breaking runtime JS. add_header in location blocks drops parent CSP.

Affected files: docker/nginx.conf, docker/nginx-ssl.conf, Dockerfile, vite.config.ts, sandbox-service.ts

Safe fix: 1) Add envsubst to Dockerfile entrypoint. 2) Add /proxy/* location blocks to nginx. 3) Add 'unsafe-eval' to CSP for the app's own scripts. 4) Use add_header with 'always' and inherit parent headers.

Original findings: BLD-01/02/06/07/08/09/12

#15 CRITICAL

4 audits

ResumableStream: resume() restarts from scratch, yields duplicate content

Root cause: ResumableStream.resume() does not actually resume; it restarts the stream from the beginning, yielding duplicate content with misleading indices. switchProvider() does not abort the old generator. Missing failed status check after pause loop.

Affected files: resumable-stream.ts

Safe fix: 1) Track last yielded chunk index. 2) On resume, skip already-yielded chunks. 3) Abort old generator in switchProvider(). 4) Check state.status === 'failed' between attempts.

Original findings: RC-16/19/52, L-02/05/20, CV-20/26, TC-41

#16 HIGH

3 audits

Circuit breaker: stale success resets failure counter while circuit is open

Root cause: A late-arriving success from a previous request resets failures to 0 even while the circuit is in OPEN state, incorrectly closing the circuit. Dead-code no-op inFlightHalfOpen reset. Stale onSuccess callback from a half-open test can leak through.

Affected files: circuit-breaker.ts

Safe fix: 1) Only reset failure counter on success if circuit is CLOSED or HALF_OPEN and the request was initiated in that state. 2) Remove dead-code inFlightHalfOpen reset. 3) Add request ID tracking to match success/failure pairs.

Original findings: RC-18, CV-19, D-13

#17 CRITICAL

3 audits

MemoryRepository.upsert() is non-deterministic: always inserts, never updates

Root cause: computeId() produces different IDs for the same logical entity on each call (uses timestamps or random values). This means upsert always creates duplicates instead of updating existing records, silently corrupting data.

Affected files: dal/memory-repository.ts, memory-engine.ts

Safe fix: Make computeId() deterministic: use a stable key derived from the entity's natural key (e.g., userId+memoryId). Add upsert-by-key semantics instead of always-insert.

Original findings: CV-01, D-07/08, LG-61

#18 CRITICAL

2 audits

Schema/migration drift: Dexie versions diverge, data lost on upgrade

Root cause: Multiple services define their own Dexie schema versions that conflict. Migration functions silently drop fields. database-service.ts and debate-session-persistence.ts have incompatible version schemes. No migration validation or rollback.

Affected files: database-service.ts, debate-session-persistence.ts, migration-control-layer.ts, memory-engine.ts

Safe fix: 1) Centralize all Dexie schema definitions into database-service.ts. 2) Add migration validation (field count, type checks). 3) Add rollback capability for failed migrations. 4) Log all migration steps.

Original findings: D-02/03/04/18/19/29/39/40, CV-02

#19 HIGH

3 audits

SSE parser data accumulator lost across pull() calls, multi-line events broken

Root cause: The SSE parser's dataAccumulator variable is not preserved across pull() calls in the ReadableStream, losing partial data for multi-line SSE events. Consecutive data: fields are joined without a newline, violating the SSE specification.

Affected files: sse-parser.ts

Safe fix: 1) Store dataAccumulator in a class property instead of a local variable. 2) Join consecutive data: fields with newline per SSE spec. 3) Add integration tests for multi-line SSE events.

Original findings: RC-03, LG-36/37

#20	HIGH	3 audits	Service destroy() methods missing or incomplete: 10+ services leak on shutdown
Root cause: CrossTabStateSync, DebateRuntimeAdapter, BrowserSTTService, ProxyHealthMonitor, ProviderRuntimeService, CacheService, DebateBudget, and others either have no destroy() or their destroy() doesn't clean up timers, listeners, and subscriptions. Affected files: cross-tab-state.ts, debate-runtime-adapter.ts, browser-stt.ts, proxy-health-monitor.ts, provider-service.ts, cache-service.ts, budget-service.ts Safe fix: 1) Add destroy() to all services that hold timers, listeners, or subscriptions. 2) Register all services with LifecycleManager so shutdown calls destroy() in reverse order. 3) Add a destroy() audit checklist. Original findings: RC-09/13/14/15/29/33, CV-09/10/11/32/33/34/35			
#21	CRITICAL	3 audits	ChatService.destroy() does not abort in-flight requests; stale mutations after destroy
Root cause: ChatService.destroy() unsubscribes from events but does not abort active streaming requests. After destroy, in-flight request callbacks can still mutate state, causing errors or corruption. Affected files: chat-service.ts Safe fix: Store AbortController for each active request. In destroy(), abort all controllers before unsubscribing. Original findings: L-03, RC-14, CV-09			
#22	CRITICAL	2 audits	UsageTracker.checkQuota() always succeeds: quota enforcement is non-functional
Root cause: checkQuota() returns true regardless of actual usage. This means budget limits are completely ignored: any user can exceed any quota without restriction. Affected files: usage-tracker.ts, key-quotas.ts Safe fix: Implement actual quota checking: compare current usage against configured limits. Return false when exceeded. Add integration tests. Original findings: OBS-17, LG-11			
#23	HIGH	2 audits	Cross-tab primary election always returns true: base-36 vs base-10 parsing bug
Root cause: Primary election compares tab IDs using parseInt(id, 10) but tab IDs are base-36. This means the first tab always wins, and the 'primary' role never transfers when that tab closes. Affected files: cross-tab-state.ts Safe fix: Change parseInt(id, 10) to parseInt(id, 36) in the primary election logic. Original findings: LG-19, SI-02			
#24	HIGH	3 audits	FeatureFlagService.init()/start() are no-ops: flags never persist across reloads
Root cause: FeatureFlagService has empty init() and start() methods. Runtime flag changes are never persisted, so all feature flags reset to defaults on page reload. Mutations have zero observability. Affected files: feature-flag-service.ts Safe fix: 1) Persist flags to StorageAdapter on change. 2) Load persisted flags in init(). 3) Emit FEATURE_FLAG_CHANGED event on mutation. Original findings: TC-43, BLD-17, OBS-92			

#25	HIGH	3 audits	ConfigService.updateRouter() silently ignores its parameter: a complete no-op
Root cause: updateRouter() accepts a config argument but discards it entirely. The method returns without doing anything. getRouter() bypasses the overlay system. This means router config changes are impossible through the API. Affected files: config-service.ts Safe fix: Implement updateRouter() to apply changes via the overlay system and emit CONFIG_CHANGED. Original findings: CV-28, TC-37, BLD-31			
#26	HIGH	2 audits	DebateSession.transition() silently swallows invalid state transitions
Root cause: When an invalid transition is attempted (e.g., paused->completed), transition() returns false but doesn't log, emit, or throw. The caller has no way to know the transition failed. VALID_TRANSITIONS is also missing paused->queued, breaking resume. Affected files: debate-session.ts Safe fix: 1) Add paused->queued to VALID_TRANSITIONS. 2) Throw or emit event on invalid transition. 3) At minimum, log the attempted transition. Original findings: CV-18, TC-39			
#27	CRITICAL	2 audits	KeyHealth returns phantom 'healthy' for missing/unknown keys
Root cause: When checkHealth() is called for a key that doesn't exist in KeyStateStore, it returns a 'healthy' result instead of 'unknown' or 'missing'. This creates phantom healthy signals that mislead routing and UI. Affected files: key-health.ts, key-state-store.ts Safe fix: Return status:'unknown' for keys not found in KeyStateStore. Treat 'unknown' as unhealthy for routing purposes. Original findings: OBS-06, CV-29			
#28	HIGH	2 audits	HealthService isRunning flag can get permanently stuck: never resets on unexpected error
Root cause: If HealthService's health check loop encounters an unexpected error (not a per-key error but a structural error), isRunning stays true but the loop stops. No future health checks will ever run. Affected files: health-service.ts Safe fix: Wrap the health check loop in try/catch/finally. In finally, always reset isRunning to false. Add a watchdog timer that detects stale health data. Original findings: OBS-38, RC-42			
#29	CRITICAL	2 audits	TaskQueue permanently stalls when throttle is enabled
Root cause: When throttleMs is set, the queue's processNext() method awaits a throttle delay but then re-enters itself recursively without proper guard, causing the queue to stall permanently. Affected files: core/TaskQueue.ts Safe fix: Replace recursive processNext() with a loop-based approach. Ensure throttle delay is followed by processing, not by re-queuing without processing. Original findings: LG-04, RC-26			

#30

CRITICAL

2 audits

RaceExecutor resolves to failure if fastest candidate fails, ignoring slower successes

Root cause: In Promise.race-style execution, if the fastest candidate rejects, the entire race fails even if slower candidates would succeed. This wastes the successful results and is not true 'racing' behavior.

Affected files: race-executor.ts

Safe fix: Use Promise.any() instead of Promise.race() for the success path. Only reject if ALL candidates fail.

Original findings: LG-03, RC-40

#31

HIGH

2 audits

Transaction.commit() resets _committed during failure, allowing interleaved mutations

Root cause: If commit() fails partway, it sets _committed=false, allowing the transaction to be retried. But other code may have already seen _committed=true and acted on it, creating inconsistency.

Affected files: transaction.ts

Safe fix: 1) Set _committed=true only after all persistence succeeds. 2) On failure, mark the transaction as permanently failed, not reusable. 3) Add a separate _failed flag.

Original findings: LG-23, CV-45

#32

CRITICAL

2 audits

ICacheService/IHealthService/IMemoryEngine contracts not implemented by concrete services

Root cause: ICacheService declares a clear() method that CacheService doesn't implement. IHealthService has methods that HealthService doesn't implement. IMemoryEngine is missing 4+ methods. The wrong class is registered as healthCheckService.

Affected files: contracts/cache.ts, contracts/health.ts, contracts/memory.ts, phase4-agents-roles.ts

Safe fix: 1) Audit all contract interfaces vs implementations. 2) Either implement missing methods or remove them from interfaces. 3) Fix phase4 registration to use the correct class.

Original findings: TC-14/15/17/18

#33

HIGH

2 audits

SnapshotService.restore() does not invalidate caches or projections

Root cause: After restoring a snapshot, all derived state (caches, projections, metrics) still reflects the old state. The system appears restored but behaves with stale data.

Affected files: snapshot-service.ts, cache-service.ts, projection-registry.ts

Safe fix: After restoring snapshot data, call: 1) CacheInvalidator.invalidateAll(). 2) ProjectionRegistry.rebuildAll(). 3) Emit SNAPSHOT_RESTORED event.

Original findings: SI-04, LG-52

#34

HIGH

2 audits

PricingService hardcodes costs instead of using the pricing service for calculations

Root cause: key-analytics.ts uses hardcoded \$0.01/M-token for cost calculation. PricingService.lastFetch=0 cannot distinguish 'never synced' from 'synced at epoch'. checkProviderBudget uses startsWith for matching, causing false positives.

Affected files: key-analytics.ts, pricing-service.ts

Safe fix: 1) Use PricingService.getInstance().getModelPricing() instead of hardcoded values. 2) Set lastFetch = null (not 0) when never synced. 3) Fix provider matching to use exact comparison.

Original findings: LG-08/09/34, OBS-36

#35

CRITICAL

2 audits

Budget threshold double-counts current request cost, reporting false over-budget

Root cause: In the `STREAM_END` handler, `calculateCost()` is called and the result is added to spending. But the cost was already added during request initialization. This double-counting causes the budget to appear exceeded prematurely.

Affected files: `budget-service.ts`, `key-quotas.ts`

Safe fix: Track whether cost has already been counted for the current request. Only add cost once, either at start or at end, not both.

Original findings: LG-01, SI-25

#36

CRITICAL

1 audit

ErrorBoundary catches errors but doesn't report to any monitoring service

Root cause: The React `ErrorBoundary` catches render errors and shows a fallback UI, but never sends the error to any monitoring, logging, or event system. Production crashes are invisible to operators.

Affected files: `components/Common/ErrorBoundary.tsx`

Safe fix: Add an error reporting call in `componentDidCatch()`: emit to `EventBus`, send to logging service, or at minimum call `console.error` with full stack.

Original findings: OBS-71

#37

CRITICAL

1 audit

Container.clear() silently swallows all destroy() errors, hiding startup/shutdown failures

Root cause: When the DI container is cleared (e.g., during shutdown), all service `destroy()` calls are wrapped in individual `try/catch` that silently swallow errors. A service that fails to clean up (e.g., leaving database connections open) goes undetected.

Affected files: `kernel/container.ts`

Safe fix: Log each `destroy()` error. Collect all errors and emit a `CONTAINER_DESTROY_FAILED` event with the error list. In dev mode, throw on any destroy failure.

Original findings: OBS-91

#38

CRITICAL

1 audit

Zustand stores mutate state with zero observability: no hooks, no events, no logging

Root cause: `debateLiveStore`, `topologyTraceStore`, `useKeyStore`, and `useSystemStatus` all mutate state directly with no middleware for logging, event emission, or change tracking. State changes are completely invisible to the kernel's event system.

Affected files: `debateLiveStore.ts`, `topologyTraceStore.ts`, `useKeyStore.ts`, `useSystemStatus.ts`

Safe fix: 1) Add Zustand devtools middleware for development. 2) Add `subscribeWithSelector` middleware. 3) For critical stores, add a bridge that emits key state changes to the `EventBus`.

Original findings: OBS-73/75/76

#39

HIGH

1 audit

ProxyHealthMonitor performCheck() can overlap when fetch exceeds interval

Root cause: If a health check fetch takes longer than the check interval, the next check starts while the previous one is still running. This can cause stale mutations after destroy and conflicting state updates.

Affected files: `proxy-health-monitor.ts`

Safe fix: Add a `mutex/guard`: only start a new check if the previous one has completed. If the previous is still running, skip or queue the new check.

Original findings: RC-17

#40	HIGH	1 audit	debounce/throttle utilities have no cancel/flush and stale closure risk
Root cause: The debounce() utility has no cancel() or flush() methods, making cleanup impossible. throttle() drops all calls during the window with no trailing edge. Both capture stale closures that can reference outdated state. Affected files: utils/debounce.ts Safe fix: 1) Add cancel() and flush() methods to debounce. 2) Add trailing edge option to throttle. 3) Document that callers must cancel on unmount. Original findings: RC-24/25			
#41	CRITICAL	1 audit	Cancelled responses stay in loading state forever in useChatStore
Root cause: When the user clicks stop during streaming, cancelSending sets isSending=false but doesn't update the message status from 'loading' to 'cancelled'. The UI shows the message as permanently loading. Affected files: stores/useChatStore.ts Safe fix: In cancelSending(), set the message status to 'cancelled' or 'stopped' in addition to setting isSending=false. Original findings: UX-01			
#42	HIGH	2 audits	Hardcoded localhost fallbacks in sandbox/tool-executor break in Docker/production
Root cause: SandboxService, ToolExecutor, and OpenRouter adapter all fall back to localhost:5173 or localhost:3001 when environment variables are missing. In Docker, localhost refers to the container, not the host, breaking all proxy functionality. Affected files: sandbox-service.ts, tool-executor.ts, openrouter-adapter.ts, cors-proxy.mjs Safe fix: 1) Remove localhost fallbacks. 2) Throw or show a clear error when required env vars are missing. 3) Use Docker host.docker.internal or configurable hostnames. Original findings: BLD-07/12/16/21/28/29			
#43	CRITICAL	1 audit	EventsPanel crashes at runtime: SEVERITY_CONFIG and TYPE_COLORS undefined
Root cause: The EventsPanel component references SEVERITY_CONFIG and TYPE_COLORS objects that are never defined in scope, causing a runtime crash when the panel is opened. Affected files: EventsPanel.tsx Safe fix: Define the missing SEVERITY_CONFIG and TYPE_COLORS constants, or import them from a shared theme file. Original findings: UX-03/04			
#44	CRITICAL	2 audits	Dual KeyStatus/KeyState enum with conflicting values across files
Root cause: KeyStatus in contracts/key-state.ts uses 'ready' 'error' ... while KeyState in types/metrics-types.ts uses 'active' 'degraded' ... Code casts between them with 'as KeyStatus', causing semantic mismatches. Affected files: contracts/key-state.ts, types/metrics-types.ts, key-state-projection.ts Safe fix: Unify into a single KeyStatus enum. Remove the duplicate KeyState. Update all casts to use the canonical enum. Original findings: TC-12, CV-04, SI-01			

#45	MEDIUM	1 audit	buildId hardcoded as 'a9f3b2c': every build claims to be the same version
Root cause: ConfigRegistry hardcodes buildId to 'a9f3b2c'. There is no mechanism to inject the actual build hash. This makes it impossible to verify which version is deployed, undermining debugging and rollback decisions. Affected files: config-registry.ts Safe fix: Inject buildId at build time via VITE_BUILD_ID environment variable. Default to 'dev' in development. Original findings: BLD-20			
#46	HIGH	1 audit	Settings merge is shallow: new default fields in nested objects lost for existing users
Root cause: When SettingsService merges saved settings with new defaults (e.g., after an upgrade), it uses shallow merge. Any new nested fields in the defaults are silently dropped if the user has any saved value for the parent key. Affected files: settings-service.ts Safe fix: Use deep merge that preserves user overrides while adding new default fields. Add a migration step for each settings version bump. Original findings: BLD-18			
#47	HIGH	1 audit	Dexie DB instantiated at module scope crashes SSR and test environments
Root cause: DatabaseService creates the Dexie database at module import time (top-level scope). In SSR or test environments where IndexedDB doesn't exist, this crashes immediately on import, before any code can intercept. Affected files: database-service.ts Safe fix: Move Dexie instantiation into a lazy init() method. Add an isBrowser() guard. Only create the database when first accessed. Original findings: BLD-05			
#48	HIGH	1 audit	Streaming adapterMeta overwrites accumulated content instead of appending
Root cause: In llm-client-service.ts, the STREAM_DATA handler assigns adapterMeta instead of merging it, overwriting any previously accumulated metadata during streaming. Affected files: llm-client-service.ts Safe fix: Change the assignment to a merge: Object.assign(existing, incoming) or spread operator. Original findings: LG-25			
#49	MEDIUM	1 audit	SSE consecutive data: fields joined without newline: spec violation
Root cause: Per the SSE specification, consecutive 'data:' fields should be joined with a newline (U+000A). The current implementation joins them without any separator, corrupting multi-line data. Affected files: sse-parser.ts Safe fix: Add '\n' between consecutive data: field values, per the SSE specification. Original findings: LG-37			

#50

MEDIUM

2 audits

ProviderBudget.endSession decrements sessionCount: losing historical record

Root cause: endSession decrements providerSessionCount, which means the counter only reflects currently active sessions, not total sessions. This also semantically conflicts with the counter's name and breaks capacity planning metrics.

Affected files: provider-budget.ts

Safe fix: Separate activeSessionCount (decremented on end) from totalSessionCount (never decremented). Use activeSessionCount for capacity, totalSessionCount for analytics.

Original findings: OBS-40, LG-32